

Software Project Management

- Software project management is a proper way of planning and leading software projects.
- Software projects are planned, implemented, monitored, and controlled within variables such as time, quality, and cost.
- Software project management is a subset of project management, which is the application of knowledge, skills, tools, and techniques to achieve specific goals and meet specific success criteria.
- Software project management involves the following activities:
 - Gathering client requirements
 - Building software products
 - Testing their functionalities and usability
 - Preparing documentation
 - Delivering and maintaining the software
- Software project management requires the use of project management software, which is software used for project planning, scheduling, resource allocation, and change management.
- Software project management also requires the role of a software project manager, who is a liaison between the development team and the other stakeholders in a software project.
- Software project managers are responsible for:
 - Communicating project status
 - Managing changes and risks
 - Requesting additional resources
 - Ensuring quality standards
 - Resolving conflicts and issues
- Software project management follows a life cycle, which is a series of phases that a software project goes through from initiation to closure.
- Software project management life cycle models include:
 - Waterfall model: a linear and sequential approach that follows a fixed set of steps and deliverables
 - Agile model: an iterative and incremental approach that adapts to changing requirements and feedback
 - Spiral model: a risk-driven approach that combines elements of waterfall and agile models
 - V-model: a verification and validation approach that maps each development stage to a corresponding testing stage
 - Prototyping model: an experimental approach that creates a working model of the software before developing the final product
- Software project management best practices include:
 - Defining the project scope and objectives clearly
 - Estimating the project cost and duration realistically
 - Allocating the project resources efficiently
 - Developing a project plan and schedule
 - Monitoring and controlling the project progress and performance

- Managing the project quality and risks
- Communicating and collaborating with the project team and stakeholders
- Reviewing and evaluating the project outcomes and lessons learned

Unit - 1 - Project Evaluation and Project Planning

- Project evaluation is the process of assessing the feasibility, desirability, and viability of a proposed software project.
- Project planning is the process of defining the scope, objectives, tasks, resources, schedule, budget, and quality requirements of a software project.
- Project evaluation and planning are essential for software project management, as they help to align the expectations of the stakeholders, identify the risks and opportunities, and establish a realistic and achievable plan for project execution and control.
- Project evaluation and planning involve the following steps:
 - **Project initiation:** This is the first step of project evaluation and planning, where the project idea or problem is identified and analyzed, and the project sponsor or client is engaged. The project initiation document (PID) or project charter is prepared, which summarizes the project background, purpose, scope, objectives, assumptions, constraints, and success criteria.
 - **Project feasibility study:** This is the second step of project evaluation and planning, where the technical, economic, social, legal, and environmental aspects of the project are evaluated to determine if the project is feasible and worth pursuing. The feasibility study report (FSR) or business case is prepared, which presents the project rationale, benefits, costs, risks, and alternatives.
 - **Project estimation:** This is the third step of project evaluation and planning, where the size, effort, duration, and cost of the project are estimated using various techniques, such as analogy, expert judgment, parametric, or function point analysis. The estimation accuracy and confidence level are also assessed and documented. The estimation report or plan is prepared, which provides the basis for project budgeting and scheduling.
 - **Project scheduling:** This is the fourth step of project evaluation and planning, where the project tasks and activities are identified, sequenced, and assigned to resources, and the project timeline and milestones are determined. Various tools and techniques, such as work breakdown structure (WBS), network diagram, Gantt chart, or critical path method (CPM) are used for project scheduling. The project schedule or plan is prepared, which shows the start and end dates, dependencies, and durations of the project tasks and activities.

- **Project budgeting:** This is the fifth step of project evaluation and planning, where the project costs are estimated, allocated, and controlled. Various tools and techniques, such as cost breakdown structure (CBS), cost baseline, or earned value management (EVM) are used for project budgeting. The project budget or plan is prepared, which shows the planned and actual costs, variances, and performance indicators of the project.
- **Project quality planning:** This is the sixth step of project evaluation and planning, where the quality standards, criteria, and metrics for the project deliverables and processes are defined and agreed upon. Various tools and techniques, such as quality management system (QMS), quality assurance (QA), quality control (QC), or quality audit are used for project quality planning. The project quality plan is prepared, which describes the quality objectives, policies, procedures, roles, and responsibilities of the project.
- **Project risk planning:** This is the seventh step of project evaluation and planning, where the potential risks and uncertainties that may affect the project are identified, analyzed, prioritized, and mitigated. Various tools and techniques, such as risk register, risk matrix, risk response plan, or Monte Carlo simulation are used for project risk planning. The project risk plan is prepared, which outlines the risk sources, impacts, probabilities, responses, and contingencies of the project.
- **Project communication planning:** This is the eighth step of project evaluation and planning, where the information needs, expectations, and preferences of the project stakeholders are identified, and the communication methods, channels, frequency, and format are determined. Various tools and techniques, such as communication matrix, stakeholder analysis, or communication plan are used for project communication planning. The project communication plan is prepared, which specifies the communication objectives, messages, media, and feedback mechanisms of the project.
- **Project procurement planning:** This is the ninth step of project evaluation and planning, where the external resources, such as goods, services, or contractors, that are required for the project are identified, and the procurement strategy, process, and contract type are decided. Various tools and techniques, such as make-or-buy analysis, request for proposal (RFP), or contract management are used for project procurement planning. The project procurement plan is prepared, which defines the procurement scope, requirements, criteria, and terms and conditions of the project.
- **Project integration planning:** This is the tenth and final step of project evaluation and planning, where the various project plans and documents are integrated, reviewed, approved, and updated. Various tools and techniques, such

Importance of Software Project Management

Software project management is the process of planning, organizing, executing, monitoring, and controlling software development projects. It involves applying knowledge, skills, tools, and techniques to achieve specific project objectives within the given constraints of scope, time, cost, and quality. Software project management is important for several reasons, such as:

- It helps to align the software development activities with the business goals and expectations of the stakeholders. Software project management ensures that the software product meets the needs and requirements of the end-users and delivers value to the organization.
- It helps to manage the complexity and uncertainty of software development projects. Software project management provides a structured framework and methodology to deal with the various challenges and risks involved in software development, such as changing requirements, technical issues, human factors, and external dependencies.
- It helps to improve the efficiency and effectiveness of software development teams. Software project management enables the teams to collaborate and communicate better, to coordinate and integrate their work, to track and measure their progress and performance, and to learn and improve from their experiences.
- It helps to ensure the quality and reliability of the software product. Software project management involves applying various quality assurance and quality control techniques, such as testing, verification, validation, and review, to ensure that the software product meets the predefined standards and specifications and satisfies the customer expectations.
- It helps to optimize the use of resources and minimize the waste and rework. Software project management helps to plan and allocate the resources (such as time, money, people, and equipment) needed for the software development project, to monitor and control the costs and schedule, and to identify and eliminate the sources of inefficiency and error.

Activities in Software Project Management

Software project management is the process of leading, organizing, and controlling a software development project. It involves a range of activities, such as:

- **Scope management:** defining and managing the scope of the software product, including the features, functions, and quality requirements.
- **Project planning and tracking:** developing and maintaining a project plan that outlines the tasks, milestones, deliverables, dependencies, resources, and schedule of the project. Tracking the progress and performance of the project against the plan and adjusting it as needed.
- **Project resource management:** allocating and managing the human, financial, and physical resources of the project, such as the team members,

budget, equipment, and tools.

- **Scheduling management:** creating and managing the project schedule, which defines the sequence and duration of the project activities and the deadlines for the deliverables.
- **Project communication management:** planning and executing the communication strategy of the project, which includes the methods, frequency, and channels of communication among the project stakeholders, such as the clients, users, team members, and managers.
- **Estimation management:** estimating the cost, effort, time, and quality of the project, using various techniques and tools, such as historical data, expert judgment, analogy, and parametric models.
- **Project risk management:** identifying, analyzing, and managing the potential risks and uncertainties that may affect the project, such as technical, operational, organizational, and external risks. Developing and implementing risk mitigation and contingency plans.
- **Project configuration management:** establishing and maintaining the consistency and integrity of the project artifacts, such as the requirements, design, code, and documentation. Controlling the changes and versions of the project artifacts and ensuring their traceability and compatibility.

Methodologies in Software Project Management

- A methodology is a set of principles, tools and techniques that are used to plan, execute and manage projects.
- A methodology helps project managers lead team members and manage work while facilitating team collaboration.
- There are many different methodologies, and they all have pros and cons.
- Some of the most popular methodologies in software project management are:
 - **Agile:** Agile is a flexible and iterative approach that focuses on delivering value to customers and responding to changes quickly. Agile projects are divided into short cycles called sprints, where teams work on small increments of the product and get feedback from stakeholders. Agile methodologies include Scrum, Kanban, Extreme Programming, and others .
 - **Waterfall:** Waterfall is a linear and sequential approach that follows a predefined set of phases, such as requirements, design, development, testing, and deployment. Waterfall projects are planned in advance and have clear deliverables and milestones. Waterfall is suitable for projects with stable and well-defined requirements and minimal changes .
 - **Scrum:** Scrum is one of the most widely used agile methodologies. Scrum projects are managed by a self-organizing and cross-functional team, a product owner who represents the customer, and a scrum

master who facilitates the process. Scrum teams work in sprints of 2-4 weeks, where they deliver potentially shippable product increments and conduct daily meetings, sprint planning, sprint review, and sprint retrospective .

- **PRINCE2:** PRINCE2 stands for Projects in Controlled Environments. PRINCE2 is a structured and process-based methodology that covers the entire project lifecycle, from initiation to closure. PRINCE2 projects are divided into manageable stages, each with clear objectives, roles, and responsibilities. PRINCE2 is suitable for projects with multiple stakeholders and complex risks .
- **Lean:** Lean is a methodology that aims to eliminate waste and maximize value in software development. Lean principles include optimizing the flow of work, delivering fast and frequently, empowering teams, building quality in, and learning from feedback. Lean methodologies include Lean Startup, Lean Software Development, and others .
- **Six Sigma:** Six Sigma is a methodology that focuses on improving the quality and efficiency of software development by reducing defects and variations. Six Sigma uses a data-driven and statistical approach to measure and analyze the performance of processes and products. Six Sigma projects follow a five-phase cycle called DMAIC: Define, Measure, Analyze, Improve, and Control .
- **Extreme Programming:** Extreme Programming (XP) is an agile methodology that emphasizes customer satisfaction, teamwork, and code quality. XP practices include frequent releases, test-driven development, pair programming, collective ownership, continuous integration, and others .
- **Critical Path Method:** Critical Path Method (CPM) is a methodology that helps project managers identify the longest sequence of tasks that determines the duration of the project. CPM also helps project managers optimize the schedule, allocate resources, and monitor progress. CPM uses a network diagram to visualize the dependencies and the critical path of the project .
- **Gantt Chart Method:** Gantt Chart Method is a methodology that uses a graphical tool to plan, schedule, and track the progress of the project. A Gantt chart shows the start and end dates, the duration, the dependencies, and the status of each task in the project. Gantt charts help project managers communicate the scope, timeline, and deliverables of the project to stakeholders .
- **Kanban:** Kanban is an agile methodology that visualizes the flow of work and limits the amount of work in progress. Kanban uses a board with columns and cards to represent the stages and tasks of the project. Kanban helps project teams improve efficiency, reduce waste, and respond to changes .
- **PMBOK:** PMBOK stands for Project Management Body of Knowledge. PMBOK is not a methodology, but a set of standards and best

practices for project management. PMBOK covers 10 knowledge areas and 5 process groups that guide project managers in planning, executing

Categorization of Software Projects in SPM

Software Project Management (SPM) is a sub-field of Project Management in which software projects are planned, implemented, monitored and controlled . Software projects can be categorized based on different criteria, such as:

- **By Development Model:** This refers to the methodology or framework used to guide the software development process. Some common development models are Agile, Waterfall, Iterative, and Incremental.
- **By Platform:** This refers to the type of hardware and software environment in which the software product will run. Some common platforms are Desktop, Mobile, Web-based, and Embedded.
- **By Scope:** This refers to the size and complexity of the software project, which can affect the cost, duration, and quality of the project. Some common scopes are Small, Medium, and Large.
- **By User Interaction:** This refers to the mode of communication between the software product and its users. Some common user interactions are Batch and Interactive.
- **By Maturity Level:** This refers to the stage of development or evolution of the software product. Some common maturity levels are New, Legacy, and Mature.
- **By Open-Source Software Participation:** This refers to the extent to which the software project involves or depends on open-source software components or communities. Some common levels of participation are None, Partial, or Full.
- **By Delivery Method:** This refers to the way the software product is deployed or distributed to its end-users. Some common delivery methods are Internal, Outsource, Hosted, or Cloud-Based.

These categories are not mutually exclusive and can be combined or modified to suit the specific needs and characteristics of each software project. Categorizing software projects can help in identifying the goals, requirements, risks, and best practices of each project.

Setting Objectives in SPM

- SPM stands for Strategic Project Management, which is a process of planning, prioritizing, and executing projects to achieve business objectives.
- Setting objectives in SPM is a crucial step to ensure that the projects are aligned with the strategic direction and goals of the organization .
- Some of the benefits of setting objectives in SPM are:
 - It gives clarity of expectations to employees
 - It focuses, motivates and engages staff

- It enables performance measurement and feedback
- It supports career development and succession planning
- Some of the steps to setting objectives in SPM are:
 - Define the strategic direction and vision of the organization
 - Identify the key business outcomes and priorities
 - Break down the outcomes into specific, measurable, achievable, relevant, and time-bound (SMART) objectives
 - Assign the objectives to the relevant projects and teams
 - Communicate the objectives and expectations to the stakeholders
 - Monitor and review the progress and performance of the objectives
 - Adjust and update the objectives as needed
- Some of the tools and techniques to support setting objectives in SPM are :
 - Project portfolio management, which is a process of selecting and managing a group of projects that support the strategic objectives
 - Resource management, which is a process of allocating and optimizing the human and material resources for the projects
 - Value management, which is a process of measuring and delivering the value of the projects to the business
 - Agile and hybrid methodologies, which are flexible and adaptive approaches to project management that can accommodate changing objectives and requirements

Management Principles in SPM

SPM stands for Software Project Management or Strategic Portfolio Management, depending on the context. Both are related to the effective planning, execution, and control of projects or portfolios that align with the organizational goals and objectives. Here are some of the common management principles in SPM:

- **Define and document requirements:** This principle involves identifying the needs and expectations of the stakeholders, the scope and objectives of the project or portfolio, and the deliverables and acceptance criteria. A clear and detailed requirements document helps to avoid ambiguity, confusion, and conflicts, and serves as a basis for planning, estimation, and evaluation.
- **Manage changes effectively:** This principle involves establishing a change management process that defines how changes to the requirements, scope, schedule, budget, or resources are requested, reviewed, approved, implemented, and communicated. A change management process helps to minimize the impact of changes on the project or portfolio performance, quality, and risk, and ensures that the changes are aligned with the strategic goals.
- **Prioritize projects:** This principle involves ranking the projects or portfolios based on their strategic value, urgency, feasibility, and dependencies.

A prioritization process helps to allocate the limited resources and budget to the most important and beneficial projects or portfolios, and to avoid overcommitment, duplication, or waste .

- **Maintain quality:** This principle involves ensuring that the project or portfolio deliverables meet the quality standards and expectations of the stakeholders, and that the project or portfolio processes are efficient, effective, and compliant. A quality management process helps to prevent, detect, and correct defects, errors, or deviations, and to improve the performance and satisfaction of the project or portfolio team and customers.
- **Utilize automation:** This principle involves using software tools and technologies to automate, streamline, or simplify the project or portfolio tasks, activities, or processes. Automation helps to reduce the manual effort, time, and cost, and to increase the accuracy, consistency, and reliability of the project or portfolio outputs and outcomes.
- **Establish communication protocols:** This principle involves defining the communication channels, methods, frequency, and content for the project or portfolio stakeholders, and ensuring that the communication is clear, timely, relevant, and respectful. Communication helps to share information, knowledge, feedback, and ideas, and to build trust, collaboration, and engagement among the project or portfolio team and customers.
- **Manage risk:** This principle involves identifying, analyzing, evaluating, and treating the potential threats or uncertainties that may affect the project or portfolio objectives, deliverables, or performance. A risk management process helps to mitigate the negative impacts of risk, and to exploit the positive opportunities of risk, and to ensure that the project or portfolio is prepared for any unforeseen events or changes.
- **Utilize a systematic approach:** This principle involves following a structured, logical, and consistent methodology or framework for the project or portfolio management, such as agile, waterfall, hybrid, or custom. A systematic approach helps to provide guidance, direction, and control for the project or portfolio lifecycle, and to ensure that the project or portfolio is aligned with the best practices, standards, and principles of the industry or domain .

Management Control in SPM

- Management control in software project management (SPM) is the process of monitoring and controlling project activities to ensure the project meets its goals and stays on budget.
- It involves setting performance targets, measuring progress towards those targets, and taking corrective action to ensure the project is on track.
- Some of the techniques and tools used for management control in SPM are:
 - **Work breakdown structure (WBS):** A hierarchical decomposition of the project scope into manageable tasks and deliverables.
 - **Gantt chart:** A graphical representation of the project schedule, show-

- ing the start and end dates of each task and their dependencies.
- Earned value management (EVM): A method of measuring the project performance by comparing the actual work done, the planned work, and the budget.
- Risk management: A process of identifying, analyzing, and mitigating the potential threats and opportunities that may affect the project outcome.
- Quality management: A process of ensuring that the project deliverables meet the customer requirements and expectations.
- Change management: A process of managing the changes that may occur during the project lifecycle, such as scope changes, requirement changes, or stakeholder changes.
- Management control in SPM is essential for achieving the project objectives and delivering value to the customers and stakeholders. It helps to avoid cost overruns, schedule delays, quality issues, and scope creep. It also enables the project manager to communicate the project status and issues to the project team and the senior management.

Risk Evaluation in SPM

- Risk evaluation is the systematic process of assessing the potential risks and their impact on a software project.
- Risk evaluation is an important part of the Software Project Management (SPM) process, as it helps to identify and mitigate the threats and uncertainties that may affect the project objectives, scope, schedule, budget, quality, and stakeholders.
- Risk evaluation consists of four steps: analyzing the project environment, identifying risks, assigning risk ratings, and creating a risk management plan.
- Analyzing the project environment involves reviewing the internal and external factors that may influence the project, such as the project scope, requirements, resources, stakeholders, technology, market, regulations, etc.
- Identifying risks involves brainstorming, researching, and listing the possible sources of risk that may affect the project, such as technical, operational, organizational, financial, legal, environmental, etc.
- Assigning risk ratings involves estimating the likelihood and severity of each risk, using qualitative or quantitative methods, such as risk matrices, probability distributions, scoring models, etc.
- Creating a risk management plan involves developing strategies and actions to avoid, reduce, transfer, or accept each risk, as well as defining the roles and responsibilities, monitoring and reporting mechanisms, and contingency plans for risk management.

Strategic Program Management in Software Project Management

- Strategic program management is a centralized way to coordinate the strategic goals and objectives of a program, which is a collection of related projects that share a common vision and deliver benefits to the organization.
- Strategic program management aligns the program with the enterprise portfolio and strategic management, which are the processes of selecting, prioritizing, and monitoring the performance of the organization's initiatives and investments.
- Strategic program management helps to connect the projects within the program to the strategic business goals of the organization, and to view the interdependencies, risks, and opportunities among them .
- Strategic program management also simplifies the resource allocation, stakeholder communication, and governance of the program, by providing a holistic and integrated perspective of the program's scope, schedule, budget, quality, and outcomes.
- Strategic program management requires a high level of leadership, vision, and collaboration skills, as well as a deep understanding of the business environment, the customer needs, and the program benefits.
- Strategic program management can be applied to software project management, which is the process of leading a software project performed by a team to achieve certain goals, such as developing a new software product or service.
- Strategic program management can help software project management by ensuring that the software projects are aligned with the organization's strategy, that the software products or services deliver value to the customers and stakeholders, and that the software projects are coordinated and integrated to optimize the use of resources, time, and quality.
- Some examples of strategic program management in software project management are:
 - A software company that develops a program to create a suite of software products or services that cater to a specific market segment or customer need, such as cloud computing, e-commerce, or gaming.
 - A software company that implements a program to improve the software development processes and practices across the organization, such as adopting agile methodologies, DevOps, or quality assurance standards.
 - A software company that initiates a program to transform the software architecture and technology of the existing software products or services, such as migrating to a microservices architecture, using artificial intelligence, or enhancing security features.

Stepwise Project Planning in SPM

Stepwise project planning in software project management is the process of breaking down the development process into smaller, manageable steps. This makes it easier for the project manager to plan and track progress, as each step is clearly defined. It also allows for tasks to be divided up and delegated to different team members.

The steps involved in stepwise project planning are :

- **Step 0: Select project.** This involves choosing a project that is feasible, beneficial, and aligned with the organization's goals and strategies.
- **Step 1: Identify project scope and objectives.** This involves defining the boundaries, deliverables, and expected outcomes of the project, as well as the criteria for measuring success and quality.
- **Step 2: Identify project infrastructure.** This involves identifying the resources, roles, responsibilities, and communication channels that will support the project execution and management.
- **Step 3: Analyze project characteristics.** This involves identifying the key features, functions, and requirements of the project, as well as the risks, assumptions, and constraints that may affect it.
- **Step 4: Identify project products and activities.** This involves decomposing the project into smaller units of work, such as products, tasks, and subtasks, and defining their dependencies and relationships.
- **Step 5: Estimate effort for each activity.** This involves estimating the time, cost, and resources needed for each activity, using various techniques and tools, such as expert judgment, analogy, parametric, or bottom-up methods.
- **Step 6: Identify activity risks.** This involves identifying the potential threats and opportunities that may affect the project activities, and assessing their probability and impact.
- **Step 7: Allocate resources.** This involves assigning the available resources, such as people, equipment, and materials, to the project activities, based on their availability, skills, and preferences.
- **Step 8: Review/publicize plan.** This involves reviewing the project plan for completeness, consistency, and accuracy, and communicating it to the relevant stakeholders, such as the project team, sponsors, customers, and users.
- **Step 9: Execute plan/lower levels of planning.** This involves implementing the project plan, monitoring and controlling the project performance, and making adjustments as needed. It also involves performing lower levels of planning, such as detailed design, coding, testing, and deployment, for each project product or activity.

Stepwise project planning is an iterative and flexible process, which can be adapted to the specific needs and characteristics of each project. It helps to ensure that the project is delivered on time, within budget, and with the desired

quality and functionality.

Unit - 2 - Project Life Cycle and Effort Estimation

- A software project is a complex undertaking that involves various activities, resources, and stakeholders.
- A software project life cycle is a framework that defines the phases, tasks, and deliverables of a software project from inception to completion.
- A software project effort estimation is a process of predicting the amount of time, effort, and cost required to complete a software project based on its scope, size, and complexity.

Software Project Life Cycle

- There are different models of software project life cycle, such as waterfall, agile, iterative, spiral, etc. Each model has its own advantages and disadvantages depending on the nature, size, and requirements of the project.
- However, most software project life cycle models share some common phases, such as:
 - **Planning:** This phase involves defining the project objectives, scope, schedule, budget, risks, quality, and stakeholders. It also involves selecting the appropriate life cycle model, development methodology, and tools for the project.
 - **Analysis:** This phase involves gathering and analyzing the requirements of the project from the stakeholders, users, and customers. It also involves defining the functional and non-functional specifications, use cases, and user stories of the project.
 - **Design:** This phase involves designing the architecture, components, interfaces, data structures, algorithms, and user interfaces of the project. It also involves creating the design documents, prototypes, and mockups of the project.
 - **Implementation:** This phase involves coding, testing, debugging, and integrating the software components of the project. It also involves creating the executable, documentation, and configuration files of the project.
 - **Testing:** This phase involves verifying and validating the functionality, performance, reliability, security, and usability of the project. It also involves conducting various types of testing, such as unit, integration, system, acceptance, and regression testing of the project.
 - **Deployment:** This phase involves delivering and installing the software product to the end-users or customers. It also involves providing training, support, and maintenance of the product.
 - **Maintenance:** This phase involves fixing the defects, enhancing the features, and updating the software product according to the changing needs and feedback of the users or customers. It also involves

monitoring and evaluating the performance and quality of the product.

Software Project Effort Estimation

- Software project effort estimation is a crucial activity that affects the success and failure of a software project. It helps in planning, managing, and controlling the resources, time, and cost of a software project.
- Software project effort estimation is a challenging and uncertain task due to the inherent complexity, variability, and unpredictability of software development. It involves many factors, such as:
 - **Project scope:** The project scope defines the boundaries, objectives, deliverables, and expectations of the project. It affects the effort estimation by determining the amount and level of work required to complete the project.
 - **Project size:** The project size measures the quantity and quality of the software product to be developed. It affects the effort estimation by indicating the amount of code, data, and documentation to be produced for the project.
 - **Project complexity:** The project complexity reflects the difficulty and uncertainty of the software development. It affects the effort estimation by influencing the amount of skills, knowledge, and experience required to complete the project.
 - **Project quality:** The project quality denotes the degree of satisfaction and value of the software product to the stakeholders, users, and customers. It affects the effort estimation by specifying the amount of standards, testing, and verification to be applied to the project.
 - **Project resources:** The project resources include the human, technical, and organizational factors that support the software development. They affect the effort estimation by determining the availability, capability, and productivity of the resources for the project.
- There are different techniques of software project effort estimation, such as:
 - **Expert judgment:** This technique involves consulting one or more experts who have relevant experience and knowledge of similar projects. It is based on the intuition, opinion, and judgment of the experts.
 - **Analogous estimation:** This technique involves comparing the current project with the past or existing projects that have similar characteristics and features. It is based on the historical data, analogy, and similarity of the projects.
 - **Parametric estimation:** This technique involves using mathematical models and formulas that relate the project effort to one or more project parameters, such as size, complexity, quality, etc. It is based

on the statistical analysis, regression, and correlation of the project data.

- **Bottom-up estimation:** This technique involves breaking down the

Software Process and Process Models

- A software process is a set of activities for designing, implementing, and testing a software system.
- A software process model is an abstraction of the software process, which describes the stages and order of the activities.
- Software process models help developers plan, manage, and communicate their projects, as well as estimate costs, identify risks, and ensure quality.
- There are different types of software process models, each with its own advantages and disadvantages. Some of the common ones are:
 - Waterfall: A linear and sequential model, where each stage is completed before moving to the next one. It is simple and easy to follow, but it does not allow for changes or feedback during the development.
 - Prototyping: A model that involves creating a working prototype of the software before developing the final product. It allows for user feedback and testing, but it can be costly and time-consuming to refine the prototype.
 - Incremental: A model that divides the software into small increments, each of which is developed and delivered separately. It allows for flexibility and customer satisfaction, but it can be difficult to integrate and manage the increments.
 - Spiral: A model that combines the iterative and risk-driven aspects of prototyping and incremental models. It involves four phases: planning, risk analysis, engineering, and evaluation. It allows for risk management and adaptation, but it can be complex and expensive to implement.
 - Agile: A model that emphasizes collaboration, communication, and customer involvement. It involves short and frequent iterations, called sprints, where the software is developed and tested. It allows for rapid delivery and responsiveness, but it requires high commitment and discipline from the team and the customer.
 - Rational Unified Process (RUP): A model that is based on the best practices of software engineering. It involves four phases: inception, elaboration, construction, and transition. It allows for iterative and incremental development, as well as risk and quality management, but it can be rigid and bureaucratic to follow.
 - Extreme Programming (XP): A model that is a subset of agile methods. It involves five values: communication, simplicity, feedback, courage, and respect. It allows for frequent releases and customer

satisfaction, but it requires high skill and trust from the team and the customer.

Choice of Process Models in Software Project Management

- A software process model is an **abstraction** of the software development process that specifies the **stages** and **order** of a process.
- Software process models help to **organize, plan, execute, and control** the activities for designing, implementing, and testing a software system.
- There are many types of software process models, each with its own **advantages** and **disadvantages**. Some of the most popular and important ones are :
 - **Waterfall model**: A linear sequential model that divides the project into distinct phases, such as requirements analysis, design, implementation, testing, and maintenance. Each phase must be completed before the next one can start, and there is no overlap or iteration between phases. This model is simple, easy to follow, and suitable for well-defined and stable projects. However, it is rigid, inflexible, and does not accommodate changes or feedback easily. It also assumes that the requirements are clear and fixed from the beginning, which is often not the case in reality.
 - **Iterative model**: A model that repeats the same set of activities in a cycle, with each cycle producing a new version of the software that is incrementally improved from the previous one. This model allows for feedback, changes, and testing at each iteration, and can cope with evolving and unclear requirements. However, it can be difficult to manage, measure, and estimate the progress and quality of the software, and it can lead to scope creep and feature bloat if not controlled properly.
 - **V model**: A model that extends the waterfall model by adding a parallel set of activities for verification and validation of each phase. For example, for each phase of design, there is a corresponding phase of testing. This model ensures that the quality of the software is checked at every stage, and that the defects are detected and fixed early. However, it also inherits the drawbacks of the waterfall model, such as being rigid, inflexible, and not accommodating changes or feedback easily.
 - **Incremental model**: A model that combines the iterative and waterfall models by dividing the project into small modules or increments, each of which is developed and delivered separately using a linear sequential approach. This model allows for early delivery of some functionality, and for feedback and changes to be incorporated in the subsequent increments. However, it also requires careful planning and coordination of the modules, and it can be challenging to integrate and test the different increments together.
 - **Spiral model**: A model that combines the iterative and waterfall

models by adding a risk analysis and evaluation phase at each iteration. This model allows for flexibility, feedback, and changes, while also managing the risks and uncertainties of the project. However, it also requires a lot of expertise and experience to conduct the risk analysis and evaluation, and it can be costly and complex to implement.

- **Agile model:** A model that follows a set of principles and values that emphasize customer satisfaction, collaboration, adaptability, and delivery of working software in short and frequent iterations. This model allows for rapid and continuous delivery of value, and for responding to changing and emergent requirements. However, it also requires a high level of communication, trust, and discipline among the team members and stakeholders, and it can be difficult to scale, document, and maintain the software.
- The choice of a software process model for a project depends on various factors, such as :
 - The **size** and **complexity** of the project: Larger and more complex projects may require more structured and formal models, such as the waterfall or V model, while smaller and simpler projects may benefit from more flexible and informal models, such as the agile or iterative model.
 - The **requirements** and **scope** of the project: Projects with clear and stable requirements and scope may suit the waterfall or V model, while projects with vague and changing requirements and scope may suit the agile or iterative model.
 - The **quality** and **reliability** of the software: Projects that require high quality and reliability of the software, such as safety-critical or mission-critical systems, may suit the V or spiral model, while projects that can tolerate some defects or errors, such as prototyping or experimentation, may suit the agile or iterative model.
 - The **resources** and **constraints** of the project: Projects with limited resources and constraints, such as budget, time, or skills, may suit the incremental or agile model, while projects with ample resources

Rapid Application Development in Software Project Management

- Rapid Application Development (RAD) is a concept that products can be developed faster and of higher quality through:
 - Gathering requirements using workshops or focus groups
 - Prototyping and early, reiterative user testing of designs
 - The re-use of software components
 - A rigidly paced schedule that refers design improvements to the next product version
 - Less formality in reviews and other team communication
- RAD is an agile project management strategy popular in software development.

- The key benefit of a RAD approach is fast project turnaround, making it an attractive choice for developers working in a fast-paced environment like software development.
- The RAD approach prioritizes development and building a prototype, rather than planning and requirements recording.
- RAD typically involves using pre-built, reusable code libraries and frameworks and agile development techniques such as incremental development and prototyping.
- The steps of RAD are:
 - Define project requirements: Here everyone involved—you, developers, software users, and stakeholders—define, research, and agree on the scope and goals of the project.
 - Build prototypes: Next, your team begins to build out models and prototypes. The goal is to rapidly produce a working version of the software that can be tested and improved.
 - Construction, testing, and feedback: In this phase, your team iterates on the prototypes, adding features, fixing bugs, and incorporating user feedback. The aim is to deliver a functional and satisfactory product to the customer as soon as possible.
 - Finalize and launch: Once the product meets the customer’s expectations and requirements, it is ready to be deployed and launched. The team can also document the lessons learned and best practices from the project.

Agile Methods in Software Project Management

- Agile methods are a set of project management frameworks that break projects down into several dynamic phases, commonly known as sprints.
- Agile methods are based on delivering requirements iteratively and incrementally throughout the life cycle, adapting to changes and feedback from users and stakeholders.
- Agile methods are suitable for software development projects that face uncertainty, complexity and changing requirements.
- Some of the common agile methods are:
 - **Kanban:** A visual approach to agile that uses online boards to represent the workflow and progress of tasks.
 - **Scrum:** A popular agile method for small teams that involves sprints, daily meetings, and a Scrum master who facilitates the process.
 - **Extreme Programming (XP):** An agile method that focuses on delivering high-quality software through frequent releases, testing, and collaboration.
 - **Lean:** An agile method that emphasizes eliminating waste, maximizing value, and delivering fast and frequently.

- **DSDM**: An agile method that provides a comprehensive framework for delivering projects on time, on budget, and with quality.
- Agile methods have some common principles and values, such as:
 - **Customer satisfaction**: Delivering valuable software that meets the customer’s needs and expectations.
 - **Collaboration**: Working together with the customer and the team, and embracing feedback and communication.
 - **Simplicity**: Doing the simplest thing that works, and avoiding unnecessary complexity and documentation.
 - **Flexibility**: Welcoming and responding to change, and being open to new ideas and opportunities.
 - **Quality**: Ensuring that the software is reliable, usable, and maintainable, and that it meets the standards and criteria.
 - **Empowerment**: Giving the team the autonomy and responsibility to make decisions and solve problems.
 - **Continuous improvement**: Reflecting and learning from each sprint, and applying the lessons to the next one.

Dynamic System Development Method in spm

- Dynamic System Development Method (DSDM) is an agile project delivery framework that was initially used as a software development method.
- DSDM aims to deliver projects that satisfy the needs of stakeholders by involving them throughout the project lifecycle and empowering the project teams to make decisions .
- DSDM is based on eight principles that guide the project management and development process:
 - Focus on the business need: The project should deliver value to the business and align with its strategic goals.
 - Deliver on time: The project should adhere to the agreed timebox and prioritize the most important features.
 - Collaborate: The project should involve active participation and communication among all stakeholders, including users, developers, managers, and testers.
 - Never compromise quality: The project should ensure that the delivered products meet the predefined quality standards and acceptance criteria.
 - Build incrementally from firm foundations: The project should start with a clear and stable vision and scope, and then deliver functional increments based on user feedback and testing.
 - Develop iteratively: The project should use an iterative and incremental approach that allows for changes and improvements based on learning and feedback.
 - Communicate continuously and clearly: The project should use various channels and techniques to share information and knowledge

- among the project team and stakeholders.
- Demonstrate control: The project should use appropriate tools and techniques to monitor and control the project progress, risks, issues, and quality.
- DSDM consists of four phases that are repeated as needed until the project is completed:
 - Feasibility: The project team evaluates the feasibility and viability of the project based on the business need, the available resources, and the technical constraints.
 - Foundations: The project team defines the project scope, vision, objectives, and high-level requirements, and creates a prioritized list of features and a delivery plan.
 - Evolutionary Development: The project team develops and delivers the products in timeboxed iterations, each consisting of analysis, design, build, and review activities. The team also conducts testing and obtains user feedback throughout the iterations.
 - Deployment: The project team deploys the products to the live environment and ensures that they meet the business and user needs. The team also conducts post-project reviews and captures the lessons learned.
- DSDM uses a set of roles and responsibilities that are assigned to the project team and stakeholders, such as business sponsor, business visionary, project manager, technical coordinator, business analyst, solution developer, solution tester, business ambassador, business advisor, and workshop facilitator.
- DSDM also uses a set of products and deliverables that are created and updated throughout the project, such as business case, project approach questionnaire, prioritized requirements list, timebox plan, functional prototype, testing records, user documentation, and benefits assessment.
- DSDM is a flexible and adaptable framework that can be used with any project, especially those that involve complex systems and operations, and that require frequent changes and feedback. DSDM can also be integrated with other agile methods, such as Scrum and Extreme Programming (XP).

Extreme Programming in SPM

Extreme Programming (XP) is an agile software development methodology that aims to produce higher quality software and higher quality of life for the development team. It is based on five values, five rules, and 12 practices that guide the software development process. SPM stands for Software Project Management, which is the discipline of planning, organizing, and managing software projects.

Some of the main points of Extreme Programming in SPM are:

- XP values communication, simplicity, feedback, respect, and courage. These values help the team to collaborate effectively, deliver value to the customer, and embrace change.

- XP rules are the core principles that govern the XP practices. They are: planning, managing, designing, coding, and testing. These rules ensure that the team follows a consistent and disciplined approach to software development.
- XP practices are the specific techniques that the team applies to implement the XP rules. They are: the planning game, small releases, metaphor, simple design, test-driven development, refactoring, pair programming, collective ownership, continuous integration, 40-hour week, on-site customer, and coding standards. These practices help the team to deliver working software frequently, reduce complexity, improve quality, and increase productivity.
- XP is suitable for projects that have changing or unclear requirements, small or medium-sized teams, and customers who are willing to be involved in the development process. XP is not suitable for projects that have fixed or well-defined requirements, large or distributed teams, or customers who are not available or cooperative.

Managing Interactive Processes in SPM

- SPM stands for Software Project Management, which is the discipline of planning, organizing, monitoring, and controlling software projects to achieve specific goals and objectives.
- Interactive processes are those that involve frequent communication and feedback between the project stakeholders, such as the project manager, the project team, the client, the users, and the suppliers.
- Managing interactive processes in SPM requires the following steps:
 - Define the scope and objectives of the project, and identify the key stakeholders and their expectations.
 - Establish a project plan that outlines the tasks, milestones, deliverables, resources, budget, and schedule of the project.
 - Implement the project plan by executing the tasks, coordinating the activities, and managing the risks and issues that arise during the project.
 - Monitor and control the project progress by measuring the performance, quality, and outcomes of the project against the plan, and taking corrective actions when necessary.
 - Communicate and collaborate with the stakeholders regularly by providing updates, reports, feedback, and suggestions, and resolving any conflicts or misunderstandings that may occur.
 - Review and evaluate the project results by comparing the actual outcomes with the expected outcomes, and identifying the lessons learned and the best practices for future projects.
- Managing interactive processes in SPM can benefit the project by:
 - Improving the customer satisfaction and loyalty by meeting their needs and expectations, and delivering value and benefits.
 - Enhancing the team performance and motivation by fostering a cul-

ture of trust, respect, and cooperation, and providing recognition and rewards.

- Increasing the project efficiency and effectiveness by reducing the errors, rework, and waste, and optimizing the use of resources and time.
- Supporting the project innovation and adaptation by encouraging the creativity, learning, and improvement of the project processes and products.

Basics of Software Estimation in SPM

- Software estimation is the process of predicting the effort, cost, and schedule of a software project based on the available information and assumptions.
- Software estimation is important for software project management (SPM) because it helps to plan, monitor, and control the project activities and resources.
- Software estimation is challenging because software projects are complex, uncertain, and dynamic, and involve many factors and stakeholders that can affect the outcomes.
- Software estimation involves three main steps: defining the scope, selecting the estimation technique, and applying the estimation technique.
- Defining the scope means identifying and describing the software product and the project requirements, constraints, and assumptions.
- Selecting the estimation technique means choosing an appropriate method or model to estimate the effort, cost, or schedule based on the available information and the project characteristics.
- Applying the estimation technique means using the selected method or model to calculate the estimates and analyze the results and uncertainties.
- Software estimation techniques can be classified into two main categories: algorithmic and non-algorithmic.
- Algorithmic techniques use mathematical formulas or equations to compute the estimates based on some input parameters, such as size, complexity, productivity, or quality. Examples of algorithmic techniques are COCOMO, Function Point Analysis, and SLIM.
- Non-algorithmic techniques use expert judgment, analogy, or simulation to derive the estimates based on experience, intuition, or comparison. Examples of non-algorithmic techniques are Delphi, Wideband Delphi, Planning Poker, and Monte Carlo Simulation.
- Software estimation should be performed iteratively and incrementally throughout the project life cycle, as more information becomes available and the project scope and risks change.
- Software estimation should be validated and calibrated using historical data, feedback, and lessons learned from previous or similar projects.

Effort and Cost Estimation Techniques in SPM

- Effort and cost estimation are important parts of software project management (SPM) that help in planning, budgeting, and controlling a software project.
- Effort estimation is the process of predicting the amount of human resources and time required to complete a software project .
- Cost estimation is the process of predicting the monetary expenses involved in developing, testing, and maintaining a software project .
- There are various techniques for effort and cost estimation, which can be classified into three categories :
 - Expert judgment: This technique relies on the experience and intuition of experts who have worked on similar projects before. The experts provide their estimates based on their knowledge and judgment, without using any formal models or tools. Examples of this technique are Delphi method, analogy, and expert opinion .
 - Algorithmic models: This technique uses mathematical formulas or equations to calculate the effort and cost based on some input parameters, such as the size, complexity, and quality of the software. The input parameters are usually derived from historical data or measurements of the software. Examples of this technique are Function Point Analysis (FPA), Constructive Cost Model (COCOMO), and Putnam model .
 - Machine learning: This technique uses artificial intelligence and data mining techniques to learn from historical data and generate estimates based on patterns and trends. The machine learning models can be trained and validated using various algorithms, such as regression, neural networks, and decision trees. Examples of this technique are Case-Based Reasoning (CBR), Bayesian Networks, and Support Vector Machines (SVM).
- The choice of the estimation technique depends on various factors, such as the availability and quality of data, the type and size of the project, the accuracy and reliability of the estimates, and the cost and time of the estimation process .
- The estimation process is iterative and dynamic, and it should be updated and refined throughout the project lifecycle to reflect the changes and uncertainties in the project scope, requirements, and environment .

COSMIC Full Function Points in SPM

- COSMIC stands for Common Software Measurement International Consortium, which is an organization that develops and maintains a standard method for measuring software functional size.
- Software functional size is the amount of functionality that a software system provides to its users, based on the user requirements.
- COSMIC function points are a unit of measure of software functional size,

which can be used for estimating, planning, managing, and benchmarking software projects or products.

- COSMIC function points are based on the concept of data movements, which are the smallest units of functionality that can be identified in a software system.
- There are four types of data movements: Entry, Exit, Read, and Write.
- Entry: a data movement that brings data from the user domain into the software domain, such as a user input or a file upload.
- Exit: a data movement that sends data from the software domain to the user domain, such as a report, a screen display, or a file download.
- Read: a data movement that retrieves data from persistent storage within the software domain, such as a database query or a file read.
- Write: a data movement that stores data in persistent storage within the software domain, such as a database update or a file write.
- Each data movement has a size of one COSMIC function point, regardless of the complexity or volume of the data involved.
- To measure the functional size of a software system using COSMIC function points, the following steps are required:
 - Identify the functional processes of the system, which are the groups of data movements that perform a specific user requirement.
 - Identify the data groups of the system, which are the logical collections of data that are relevant to the user and the software.
 - Count the number of data movements of each type for each functional process, and assign them to the corresponding data groups.
 - Sum the number of data movements for each functional process, and then sum the number of data movements for the whole system. This is the total functional size in COSMIC function points.
- COSMIC function points can be applied to any type of software, regardless of the technology, domain, or methodology used to develop it.
- COSMIC function points can be estimated from early stages of software development, such as requirements analysis or design, or measured from later stages, such as testing or deployment.
- COSMIC function points can be used for various purposes, such as:
 - Estimating the effort, cost, and duration of software projects or products, based on historical data or industry benchmarks.
 - Planning and managing the scope, quality, and risk of software projects or products, based on the functional size and the expected or actual performance indicators.
 - Benchmarking the productivity, quality, and value of software projects or products, based on the functional size and the delivered or perceived outcomes.

COCOMO II in SPM

- COCOMO II stands for Constructive Cost Model II, which is a software cost estimation model developed at the University of Southern California

- COCOMO II is an update of the original COCOMO 81 model, which was published in 1981 .
- COCOMO II is designed to address the software development practices in the 1990s and 2000s, such as rapid application development, reuse, and object-oriented programming .
- COCOMO II consists of three sub-models: Application Composition, Early Design, and Post-Architecture .
- Application Composition model is used for estimating the effort and schedule of projects that use rapid application development or application generators .
- Early Design model is used for estimating the effort and schedule of projects in the early stages of development, when only the size and functionality are known .
- Post-Architecture model is used for estimating the effort and schedule of projects after the architecture and detailed design are completed .
- COCOMO II uses a formula to calculate the effort and schedule based on the size of the software (measured in source lines of code or function points), the project type (organic, semi-detached, or embedded), and various cost drivers (such as product, platform, personnel, and project attributes) .
- COCOMO II also provides a calibration mechanism to adjust the model parameters based on the historical data of similar projects .
- COCOMO II is a widely used and well-tested software cost estimation model that provides about 20% cost and 70% time estimate accuracy .

A Parametric Productivity Model in SPM

- A parametric productivity model is a statistical model that estimates the productivity of a software project based on some input parameters, such as size, complexity, effort, quality, etc.
- SPM stands for Statistical Parametric Mapping, which is a software package for analyzing brain imaging data, such as functional magnetic resonance imaging (fMRI), positron emission tomography (PET), or electroencephalography (EEG).
- A parametric productivity model in SPM can be used to study how different factors affect the brain activity related to software development tasks, such as coding, debugging, testing, etc.
- To construct a parametric productivity model in SPM, the following steps are required:
 - Define the software development tasks and the input parameters that are relevant for the productivity model.
 - Collect brain imaging data from software developers while they perform the tasks under different conditions or levels of the input parameters.
 - Preprocess the brain imaging data to remove noise, artifacts, and

- confounding factors, such as head motion, physiological signals, etc.
- Specify a general linear model (GLM) that relates the brain imaging data to the task conditions and the input parameters, using parametric modulation regressors to capture the variability in the neural response across different instances of the same condition.
- Estimate the GLM parameters using a maximum likelihood or a Bayesian approach, and test for significant effects of the task conditions and the input parameters on the brain activity, using statistical inference methods, such as t-tests, F-tests, or contrasts.
- Interpret the results of the parametric productivity model in terms of the brain regions and networks that are involved in the software development tasks and how they are modulated by the input parameters.

Unit - 3 - Activity Planning and Risk Management

- Activity planning is the process of defining, sequencing, and estimating the duration and resources for the tasks or activities required to complete a project.
- Risk management is the process of identifying, analyzing, evaluating, and treating the potential threats or uncertainties that may affect the project objectives, schedule, budget, or quality.
- Both activity planning and risk management are essential for successful software project management, as they help to ensure that the project is delivered on time, within budget, and meets the customer's expectations and requirements.

Activity Planning

- Activity planning involves the following steps:
 - Define the project scope and objectives, and decompose them into manageable work packages or deliverables.
 - Identify the tasks or activities needed to produce each deliverable, and their dependencies and constraints.
 - Estimate the effort, duration, and resources required for each activity, using various techniques such as expert judgment, analogy, parametric, or bottom-up methods.
 - Sequence the activities according to their logical order and dependencies, and create a network diagram or a Gantt chart to visualize the project schedule.
 - Identify the critical path, which is the longest sequence of activities that determines the minimum project duration, and the slack or float, which is the amount of time that an activity can be delayed without affecting the project completion date.
 - Review and refine the activity plan, and obtain approval from the project stakeholders.

Risk Management

- Risk management involves the following steps:
 - Plan risk management, which is the procedure of defining how to perform risk management activities for a project.
 - Identify the risks, which are the potential events or conditions that may have a positive or negative impact on the project objectives, scope, schedule, budget, or quality.
 - Analyze the risks, which is the process of assessing the probability and impact of each risk, and prioritizing them according to their severity and urgency.
 - Evaluate the risks, which is the process of determining the best response strategy for each risk, based on the cost-benefit analysis and the risk appetite of the project stakeholders.
 - Treat the risks, which is the process of implementing the risk response strategy, which can be one of the following:
 - * Avoid: eliminate the risk or change the project plan to avoid the risk exposure.
 - * Accept: acknowledge the risk and be prepared to deal with its consequences if it occurs.
 - * Reduce: take actions to reduce the probability or impact of the risk, such as testing, training, or quality control.
 - * Transfer: shift the responsibility or impact of the risk to a third party, such as insurance, contract, or warranty.
 - Monitor the risks, which is the process of tracking and reviewing the risk status, triggers, and response effectiveness, and updating the risk register and plan accordingly.

Objectives of Activity Planning in SPM

Activity planning is one of the most important management activities in software project management. It involves identifying, sequencing, estimating, and scheduling the activities that are required to complete the project. The objectives of activity planning are:

- To ensure that the appropriate resources will be available precisely when required .
- To avoid different activities competing for the same resources at the same time.
- To produce a detailed schedule showing which staff carry out each activity.
- To produce a detailed plan against which actual achievement may be measured.
- To produce a timed cash flow forecast.
- To provide a framework that enables the manager to make reasonable estimates of resources, cost, and schedule.
- To assess the feasibility of the project within the required timescales and resource constraints.

- To identify the critical path and the slack time of the project.
- To facilitate communication and coordination among the project stakeholders.
- To manage the risks and uncertainties associated with the project.

Project Schedules in SPM

- Project scheduling is the process of creating, managing, and monitoring the timeline of a software project. It involves identifying the activities, dependencies, resources, and milestones that are required to complete the project within the scope, budget, and quality constraints.
- Project scheduling is an essential part of software project management, as it helps to coordinate the work of the project team, communicate the expectations and progress to the stakeholders, and control the risks and changes that may affect the project outcome.
- Project scheduling can be done using different methods, tools, and techniques, depending on the nature, size, and complexity of the project. Some of the common methods are:
 - Critical Path Method (CPM): This method calculates the earliest and latest start and finish dates for each activity, based on their durations and dependencies. The critical path is the longest sequence of activities that determines the minimum project duration. Any delay in the critical path activities will delay the project completion.
 - Program Evaluation and Review Technique (PERT): This method estimates the duration of each activity using three scenarios: optimistic, pessimistic, and most likely. It then calculates the expected duration and variance for each activity, and uses them to determine the probability of completing the project within a given time frame.
 - Critical Chain Method (CCM): This method focuses on the availability and allocation of resources for each activity, rather than their durations. It identifies the critical chain, which is the longest sequence of resource-dependent activities, and adds buffers to protect the project from resource fluctuations and uncertainties.
- Project scheduling can be done using different tools, such as software applications, spreadsheets, or charts, that help to visualize, update, and track the project schedule. Some of the common tools are:
 - Microsoft Project: This is a popular software application that allows the project manager to create, edit, and share the project schedule, as well as assign resources, costs, and risks to each activity. It also provides various features, such as baselines, reports, and alerts, to monitor and control the project schedule.
 - OmniPlan: This is a software application designed for Mac and iOS devices, that enables the project manager to create and manage the project schedule using Gantt charts, network diagrams, calendars, and resource leveling. It also allows the project manager to collaborate with the project team and stakeholders, and sync the project

schedule across multiple devices.

- GanttPRO: This is a web-based software application that allows the project manager to create and manage the project schedule using Gantt charts, milestones, dependencies, and critical path. It also offers various features, such as templates, dashboards, and notifications, to simplify and streamline the project scheduling process.
- Project scheduling can be done using different techniques, such as decomposition, estimation, prioritization, and optimization, that help to improve the accuracy, efficiency, and effectiveness of the project schedule. Some of the common techniques are:
 - Decomposition: This technique involves breaking down the project scope into smaller and manageable units, such as deliverables, work packages, and activities, that can be easily scheduled and tracked. Decomposition helps to reduce the complexity and uncertainty of the project schedule, and increase the clarity and alignment of the project objectives.
 - Estimation: This technique involves estimating the duration, effort, and resources required for each activity, based on the available information, assumptions, and historical data. Estimation helps to determine the feasibility and realism of the project schedule, and identify the potential risks and constraints that may affect the project performance.
 - Prioritization: This technique involves ranking the activities according to their importance, urgency, and dependency, and assigning them to the project schedule accordingly. Prioritization helps to ensure that the project schedule aligns with the project scope, quality, and stakeholder expectations, and that the critical and high-priority activities are completed on time.
 - Optimization: This technique involves adjusting and fine-tuning the project schedule, based on the feedback, changes, and variances that occur during the project execution. Optimization helps to maintain the validity and reliability of the project schedule, and to enhance the project efficiency and effectiveness.

Activities in SPM

SPM stands for Software Project Management, which is a discipline that involves planning, executing, monitoring, and controlling software projects. SPM aims to deliver software products that meet the requirements of the customers, within the budget and time constraints, and with the desired quality. SPM consists of many activities that address different aspects of software development, such as:

- **Project planning and tracking:** This activity involves defining the scope, objectives, deliverables, milestones, and tasks of the project, as well as estimating the resources, cost, and duration of each task. Project planning also involves identifying and managing the risks, assumptions,

dependencies, and constraints of the project. Project tracking involves monitoring the progress, performance, and quality of the project, as well as reporting the status and issues to the stakeholders.

- **Project resource management:** This activity involves acquiring, allocating, and managing the human, physical, and financial resources of the project, such as the team members, equipment, software, and budget. Project resource management also involves motivating, leading, and communicating with the team members, as well as resolving any conflicts or problems that may arise.
- **Scope management:** This activity involves defining, validating, and controlling the features and functions of the software product, as well as the boundaries and expectations of the project. Scope management also involves managing the changes and variations that may occur in the requirements, design, or implementation of the software product, as well as ensuring that the changes are aligned with the project objectives and approved by the stakeholders.
- **Estimation management:** This activity involves estimating the size, complexity, and effort of the software product, as well as the resources, cost, and duration of the project. Estimation management also involves applying different techniques and models to produce accurate and reliable estimates, as well as adjusting and updating the estimates as the project progresses.
- **Project risk management:** This activity involves identifying, analyzing, and prioritizing the potential risks that may affect the project, such as technical, operational, organizational, or external risks. Project risk management also involves planning and implementing strategies to mitigate, avoid, transfer, or accept the risks, as well as monitoring and controlling the risks throughout the project lifecycle.
- **Scheduling management:** This activity involves defining and sequencing the activities and tasks of the project, as well as assigning the resources, dependencies, and deadlines for each task. Scheduling management also involves developing and maintaining a project schedule that shows the start and end dates, duration, and status of each task, as well as the critical path and milestones of the project.
- **Project communication management:** This activity involves planning, executing, and controlling the communication processes and channels of the project, such as meetings, reports, emails, or presentations. Project communication management also involves ensuring that the information and messages are clear, consistent, timely, and relevant to the stakeholders, as well as managing the feedback and expectations of the stakeholders.
- **Configuration management:** This activity involves identifying, organizing, and controlling the artifacts and components of the software product, such as the source code, documentation, or test cases. Configuration management also involves establishing and maintaining a repository or a database that stores and tracks the versions, changes, and status of

the artifacts and components, as well as ensuring that the artifacts and components are consistent and compatible with each other.

Sequencing and Scheduling in SPM

- SPM stands for Software Project Management, which is the process of planning, organizing, executing, and controlling software projects.
- Sequencing and scheduling are two important aspects of SPM, as they help to determine the order and timing of the project activities and resources.
- Sequencing is the process of determining which activities must be completed first, second, and so on, based on their dependencies, priorities, and constraints. Sequencing can be done using techniques such as network diagrams, critical path analysis, and precedence diagrams.
- Scheduling is the process of assigning start and finish dates, durations, and resources to each activity, based on the sequence, availability, and demand. Scheduling can be done using techniques such as Gantt charts, resource histograms, and resource leveling.
- Sequencing and scheduling are interrelated and iterative processes, as changes in one can affect the other. For example, adding or removing an activity can change the sequence and the schedule of the project.
- Sequencing and scheduling are suitable for both small and large projects, but they may require different levels of detail and complexity. For small projects, sequencing and scheduling can be combined in a single approach, such as a simple Gantt chart. For large projects, sequencing and scheduling may need to be separated and refined in multiple stages, such as a work breakdown structure, a network diagram, and a resource allocation plan.

Network Planning Models in SPM

- Network planning models are models that use graphical representations of activities and events to visualize the sequence of tasks necessary to complete the project.
- They are used to identify the duration, order of activities, and dependencies among tasks.
- They help to plan, monitor, and control the project progress and resources.
- They can also be used to analyze the impact of changes, risks, and uncertainties on the project outcome.
- Some examples of network planning models are:
 - Techamark's Planning Model: This model is focused on connecting disparate systems, such as legacy systems, cloud services, and mobile devices. It uses a network diagram to show the interfaces, protocols, and data flows among the systems. It also uses a matrix to show the compatibility, security, and performance requirements of each system.
 - Crystal's Planning Model: This model is focused on creating high-

level designs, such as architectures, modules, and components. It uses a network diagram to show the relationships, dependencies, and interactions among the design elements. It also uses a matrix to show the quality attributes, such as reliability, usability, and maintainability of each design element.

- Conway’s Planning Model: This model is focused on network reliability and scalability. It uses a network diagram to show the nodes, links, and paths of the network. It also uses a matrix to show the availability, capacity, and cost of each node and link. It also uses a formula to calculate the optimal network configuration based on the demand and budget constraints.

Formulating Network Model in SPM

- SPM stands for Software Project Management, which is the process of planning, organizing, executing, and controlling software projects.
- A network model is a graphical representation of the activities and events involved in a software project, and their dependencies and durations.
- Formulating a network model is the first stage of network planning, which is a technique for scheduling and managing software projects.
- The steps for formulating a network model are:
 1. Identify the activities and events that are required to complete the project, and their inputs and outputs.
 2. Represent the activities as nodes (boxes) and the events as circles on a graph. Use different shapes or colors to distinguish between different types of activities and events.
 3. Draw arrows between the nodes and circles to indicate the dependencies and precedence relationships among the activities and events. The direction of the arrow shows the flow of work from one node or circle to another.
 4. Assign durations and resources to each activity and event, based on the estimates and constraints of the project.
 5. Label each node and circle with a unique identifier and a descriptive name.
- There are different types of network models, such as:
 - Activity-on-node (AON), where the nodes represent the activities and the arrows represent the dependencies.
 - Activity-on-arrow (AOA), where the arrows represent the activities and the nodes represent the events.
 - Precedence diagramming method (PDM), where the nodes represent the activities and the arrows represent the dependencies, with different symbols to indicate different types of dependencies, such as finish-to-start, start-to-start, finish-to-finish, and start-to-finish.

- Graphical evaluation and review technique (GERT), where the nodes represent the activities and the arrows represent the dependencies, with probabilities and conditions attached to each arrow to indicate the uncertainty and variability of the project.
- The advantages of using a network model are:
 - It provides a clear and concise visualization of the project scope, structure, and sequence.
 - It helps to identify the critical path, which is the longest sequence of activities and events that determines the minimum duration of the project.
 - It helps to identify the slack or float, which is the amount of time that an activity or event can be delayed without affecting the project duration or the critical path.
 - It helps to monitor and control the project progress, performance, and quality, by comparing the actual and planned values of the activities and events.
 - It helps to optimize the project schedule, cost, and resources, by applying various techniques, such as crashing, fast-tracking, resource leveling, and resource allocation.

Forward Pass & Backward Pass Techniques in SPM

- SPM stands for Software Project Management, which is the process of planning, organizing, executing, and controlling software projects.
- Forward pass and backward pass are two techniques used in SPM to analyze the project network diagram and determine the project duration, critical path, and slack/float time of each activity.
- A project network diagram is a graphical representation of the sequence and dependencies of the project activities, usually using nodes to represent activities and arrows to represent dependencies.
- Forward pass is a technique to move forward through the network diagram from the start node to the end node, calculating the early start (ES) and early finish (EF) dates of each activity.
 - ES is the earliest possible date that an activity can start, given the dependencies and constraints.
 - EF is the earliest possible date that an activity can finish, calculated by adding the activity duration to the ES.
 - The forward pass starts from the start node, which has an ES of zero, and proceeds to the end node, which has an EF equal to the project duration.
 - The forward pass formula is: $EF = ES + \text{duration}$
- Backward pass is a technique to move backward through the network diagram from the end node to the start node, calculating the late start (LS) and late finish (LF) dates of each activity.
 - LS is the latest possible date that an activity can start without de-

laying the project completion date, given the dependencies and constraints.

- LF is the latest possible date that an activity can finish without delaying the project completion date, calculated by subtracting the activity duration from the LS.
- The backward pass starts from the end node, which has an LF equal to the project duration, and proceeds to the start node, which has an LS of zero.
- The backward pass formula is: $LS = LF - \text{duration}$
- The forward pass and backward pass techniques are used to identify the critical path and the slack/float time of each activity.
 - The critical path is the longest path in the network diagram, which determines the minimum project duration. It is the path with zero slack/float time for all activities.
 - The slack/float time is the amount of time that an activity can be delayed or advanced without affecting the project completion date. It is the difference between the early and late dates of an activity.
 - The slack/float time formula is: $\text{slack/float} = LS - ES = LF - EF$
 - The slack/float time can be positive, zero, or negative. A positive slack/float time means that the activity has some flexibility in its schedule. A zero slack/float time means that the activity is on the critical path and has no flexibility in its schedule. A negative slack/float time means that the activity is already behind schedule and will delay the project completion date.

Critical Path Method (CPM) in SPM

- The critical path method (CPM) is a technique where you identify tasks that are necessary for project completion and determine scheduling flexibilities .
- A critical path in project management is the longest sequence of activities that must be finished on time in order for the entire project to be complete .
- The critical path method helps you to plan, monitor, and control your project by showing you the dependencies, durations, and progress of your tasks.
- There are six steps in the critical path method :
 - Step 1: Specify Each Activity. List all the tasks that are required to complete the project and assign them an ID and a name.
 - Step 2: Establish Dependencies (Activity Sequence). Identify the logical relationships between the tasks and determine which tasks must precede or follow others.
 - Step 3: Draw the Network Diagram. Use a graphical tool to represent the tasks and their dependencies as nodes and arcs. The nodes can show the task ID, name, duration, and start and finish dates. The arcs can show the dependency type, such as finish-to-start, start-to-

start, finish-to-finish, or start-to-finish.

- Step 4: Estimate Activity Completion Time. Estimate the best-case, most likely, and worst-case durations for each task using historical data, expert judgment, or other methods. You can use the expected value formula to calculate the average duration for each task: $\text{Expected value} = (\text{Best case} + 4 \times \text{Most likely} + \text{Worst case}) / 6$
- Step 5: Identify the Critical Path. The critical path is the longest path in the network diagram, which determines the minimum time required to complete the project. To find the critical path, you need to calculate the earliest start and finish dates and the latest start and finish dates for each task using the forward pass and backward pass techniques. The tasks that have zero slack (or float) are on the critical path, meaning that any delay in them will delay the project completion.
- Step 6: Update the Critical Path Diagram to Show Progress. As the project progresses, you need to update the network diagram with the actual start and finish dates of the tasks and recalculate the critical path. You can also use the network diagram to identify the tasks that are ahead of or behind schedule, and to perform what-if analysis and risk management.
- The critical path method is a useful tool for project management, but it also has some limitations, such as:
 - It assumes that the task durations are fixed and deterministic, which may not be realistic in complex and uncertain projects.
 - It does not account for resource availability and allocation, which may affect the task durations and dependencies.
 - It focuses on the time aspect of the project, but not on the cost, quality, or scope aspects.
 - It may not identify all the critical paths in a project, especially if there are multiple paths with similar durations or if there are external dependencies.

Risk Identification in SPM

- Risk identification is the process of finding and documenting the possible sources of uncertainty that could affect the objectives of a software project .
- Risk identification is one of the most important and initial steps in risk management process, as it helps to prepare for the potential threats and opportunities that may arise during the project life cycle .
- Risk identification is an iterative process, as new risks may be identified as the project progresses, old risks may become irrelevant, and the characteristics of the existing risks may change .
- Risk identification involves brainstorming activities, where all the stakeholders meet together and generate ideas about the possible risks that could affect the project . Some of the techniques that can be used for

brainstorming are:

- SWOT analysis: This technique analyzes the strengths, weaknesses, opportunities, and threats of the project, and helps to identify the internal and external factors that could influence the project outcomes.
 - Checklist analysis: This technique uses a list of predefined questions or categories that cover the common sources of risks, such as scope, schedule, budget, quality, resources, technology, etc. The checklist can be based on historical data, industry standards, or expert opinions.
 - Cause and effect analysis: This technique uses a diagram to show the relationship between the potential causes and effects of a risk. The diagram can help to identify the root causes of a risk, and the possible consequences of a risk occurrence.
 - Delphi technique: This technique uses a panel of experts who anonymously provide their opinions on the possible risks and their likelihood and impact. The opinions are aggregated and refined until a consensus is reached.
- Risk identification should result in a risk list, which is a document that contains the description, category, source, and owner of each identified risk. The risk list should be updated and reviewed regularly throughout the project.
 - Risk identification is a crucial step for effective risk management, as it helps to plan for risk analysis, response, and monitoring. Risk identification also helps to improve the project quality, performance, and stakeholder satisfaction.

Risk Assessment in SPM

- Risk assessment is a process of identifying, analyzing, and evaluating the potential hazards and threats that may affect the objectives and outcomes of a project or an activity.
- SPM stands for Single Point Mooring, which is a system that allows oil tankers to load or unload cargo at offshore terminals without the need for anchoring or docking. SPM systems are exposed to various environmental and operational risks, such as corrosion, fatigue, collision, fire, explosion, etc.
- Risk assessment in SPM is a crucial component of the integrity management of SPM systems, which aims to ensure the safety, reliability, and performance of the systems throughout their lifecycle.
- Risk assessment in SPM involves the following steps:
 - Establishing the context: defining the scope, objectives, criteria, and stakeholders of the risk assessment.
 - Identifying the risks: listing all the possible sources and causes of failure or damage to the SPM systems and their components.
 - Analyzing the risks: estimating the likelihood and consequences of each risk, and assigning a risk level based on a predefined matrix or

scale.

- Evaluating the risks: comparing the risk levels with the acceptable or tolerable levels, and prioritizing the risks for further action.
- Treating the risks: selecting and implementing the appropriate risk mitigation or control measures, such as inspection, maintenance, repair, replacement, etc.
- Monitoring and reviewing the risks: tracking the effectiveness and performance of the risk treatment measures, and updating the risk assessment as needed.
- The benefits of risk assessment in SPM include:
 - Reducing the operational and financial losses due to SPM failures or incidents.
 - Improving the decision making and planning for SPM inspection and maintenance activities.
 - Enhancing the compliance with the regulatory and contractual requirements for SPM systems.
 - Increasing the confidence and trust of the stakeholders and customers in the SPM systems.

Risk Planning in SPM

- Risk planning is the process of identifying, analyzing, and responding to potential risks that may affect the success of a software project.
- Risk planning aims to reduce the probability and impact of negative risks, and enhance the opportunities and benefits of positive risks.
- Risk planning involves the following steps:
 - Risk identification: This is the process of determining the sources and types of risks that may affect the project, and documenting their characteristics and potential effects.
 - Risk analysis: This is the process of assessing the likelihood and consequences of each identified risk, and prioritizing them based on their severity and urgency.
 - Risk response: This is the process of developing strategies and actions to avoid, mitigate, transfer, or accept each risk, and assigning responsibilities and resources for their implementation.
 - Risk monitoring and control: This is the process of tracking, reviewing, and updating the risk status, and evaluating the effectiveness of the risk responses and adjusting them as needed.
- Risk planning is an iterative and continuous process that should be performed throughout the project life cycle, and updated as new risks emerge or change.
- Risk planning is a collaborative and participatory process that should involve all the relevant stakeholders, such as the project team, the customer,

the sponsor, the management, and the end users.

- Risk planning is a proactive and preventive process that should anticipate and address the potential risks before they become issues or problems that may jeopardize the project objectives, scope, schedule, budget, or quality.

Risk Management in SPM

- Risk management is a sequence of steps that help a software team to understand, analyze and manage uncertainty.
- Risk management consists of four main phases: risk identification, risk analysis, risk planning and risk monitoring .
- Risk identification is the process of finding and documenting the potential sources of risk that may affect the software project.
- Risk analysis is the process of estimating the probability and impact of each identified risk, and prioritizing them according to their severity.
- Risk planning is the process of developing strategies and actions to reduce, avoid, transfer or accept the risks, and allocating resources and responsibilities for implementing them.
- Risk monitoring is the process of tracking the status of the risks and the effectiveness of the risk plans, and updating them as needed.
- Risk management is one of the core tents of the philosophy of SPM, as it helps to ensure the quality, cost and schedule of the software project.
- Risk management also helps to align the software project with the strategic goals and objectives of the organization.
- Risk management is a continuous and iterative process that runs in parallel to the other PM activities throughout the project.

PERT Technique in SPM

- PERT stands for Program Evaluation and Review Technique. It is a statistical tool used in project management to analyze and represent the tasks involved in completing a given project.
- PERT was first developed by the US Navy in 1958 during the Polaris missile development program and then applied to other industries .
- PERT is used to identify task dependencies and critical paths, plan resources, estimate task duration, and identify potential risks .
- PERT helps to define and sequence activities, coordinate resources, and track progress .
- PERT uses a network diagram to represent the project activities and their interrelationships. Each activity is represented by a node and each dependency is represented by an arrow.
- PERT uses a three-point estimating technique to calculate the expected duration of each activity. The three points are optimistic, pessimistic, and most likely estimates. The expected duration is calculated as a weighted average of these three estimates .

- PERT identifies the critical path as the longest path in the network diagram. The critical path determines the minimum time required to complete the project. Any delay in the critical path activities will delay the project completion .
- PERT helps to identify the slack or float of each activity, which is the amount of time that an activity can be delayed without affecting the project completion. The slack can be used to allocate resources, manage risks, or accommodate changes.
- PERT is useful for complex and uncertain projects that involve many interdependent activities. It helps to reduce the uncertainty and improve the accuracy of the project estimates .
- PERT has some limitations, such as the difficulty of obtaining reliable estimates, the assumption of a normal distribution of activity durations, the neglect of resource constraints, and the need for frequent updates and revisions.

Monte Carlo Simulation in SPM

- SPM stands for Service Parts Management, which is the process of optimizing the availability and cost of service parts for complex products and systems.
- Monte Carlo simulation is a method of estimating the value of an unknown quantity using the principles of inferential statistics, which involves drawing random samples from a population and calculating the sample average as an estimator of the expected value.
- Monte Carlo simulation can be helpful for SPM in the following ways:
 - It can provide a time-phased picture of inventory and service levels resulting from a stocking plan, taking into account the uncertainties and variabilities in demand, supply, lead time, and failure rates.
 - It can help evaluate the trade-offs between inventory cost and service level, and find the optimal balance that minimizes the total cost of ownership and maximizes customer satisfaction.
 - It can help compare different scenarios and strategies, such as different stocking locations, replenishment policies, repair options, and sourcing alternatives, and assess their impact on the performance and profitability of the service supply chain.
 - It can help identify and mitigate the risks associated with suboptimal supply chain decisions, such as stockouts, excess inventory, obsolescence, and lost sales.

Resource Allocation in SPM

- Resource allocation in SPM (software project management) is the process of allocating, sequencing, and managing resources for a given project.
- Resources include staff, budgets, materials, and equipment needed for the successful completion of a project.

- Resource allocation in SPM is important because it gives a clear picture of the amount of work that has to be done, the time and cost required, and the progress and performance of the project team.
- Resource allocation in SPM involves the following steps:
 - Divide the project into tasks and assign them to different phases or stages of the project life cycle.
 - Estimate the effort, duration, and cost of each task and the dependencies and constraints among them.
 - Identify the available resources and their capacities, skills, and availability for each task.
 - Assign the resources to the tasks based on the priority, feasibility, and optimization of the resource utilization.
 - Monitor and control the resource allocation and adjust it as needed based on the changes in the project scope, schedule, quality, and risks.
- Resource allocation in SPM can be done using various techniques and tools, such as :
 - Resource leveling: balancing the demand and supply of resources over the project duration by delaying or splitting some tasks to avoid over-allocation or under-allocation of resources.
 - Resource smoothing: adjusting the resource allocation within the project slack or float to minimize the fluctuations or variations in the resource usage.
 - Resource histograms: graphical representations of the resource allocation over time, showing the number or percentage of resources assigned to each task or phase.
 - Resource calendars: schedules of the availability and unavailability of resources for each task or phase, taking into account the holidays, vacations, and other factors that affect the resource availability.
 - Resource management software: applications that help in planning, tracking, and optimizing the resource allocation for projects, such as ServiceNow, Microsoft Project, or TimeCamp.

Creation of Critical Paths in Project Management

- The critical path method (CPM) is a popular and effective approach to planning in project management .
- The CPM identifies the longest sequence of dependent tasks within a project that determines the minimum time required to complete the project .
- The CPM also identifies the tasks that can be completed in parallel to the critical tasks with more buffer time around them .
- The CPM allows project managers to develop realistic timelines, identify opportunities to accelerate progress, and prioritize key components .
- The CPM involves the following steps :
 - List all project tasks and break them down into milestones, deliver-

ables, and actionable tasks.

- Identify dependencies and estimates for each task, such as the predecessor, successor, duration, and resources .
 - Use a diagramming tool or software to create a network diagram that shows the sequence and relationship of tasks .
 - Calculate the earliest start, earliest finish, latest start, and latest finish for each task using the forward and backward pass method .
 - Identify the critical path by finding the tasks that have zero or negative float, meaning they have no slack time and cannot be delayed without affecting the project completion date .
 - Monitor and update the critical path as the project progresses and changes occur .
- The CPM can help project managers to improve project performance, efficiency, and quality by focusing on the most important tasks and avoiding delays and risks .

Cost Schedules in SPM

- Cost schedules are detailed plans that show the estimated costs of a software project over time.
- Cost schedules are derived from the work breakdown structure (WBS), the effort and duration estimates, and the resource allocation.
- Cost schedules are used to monitor and control the project budget, track the actual costs vs. planned costs, and identify any cost variances or issues.
- Cost schedules can be presented in different formats, such as tables, charts, graphs, or histograms.
- Cost schedules can be updated and revised throughout the project life cycle, as more information becomes available or changes occur.
- Cost schedules are part of the cost management plan, which outlines the processes and tools for estimating, budgeting, and controlling costs in a software project.